SQL Study Notes: Chapter 3 - Single Row Functions

1. Overview of SQL Functions

Oracle SQL provides a vast library of over 300 functions, with approximately 45 used frequently in standard database operations. These functions are categorized into two primary hierarchy levels:

1. **Single Row (Scalar) Functions:** These operate on individual rows and return one result per row.
2. **Multi-Row (Group) Functions:** These operate on groups of rows to return a single result.

Types of Single Row Functions

Single Row functions are classified into five distinct categories:

- **Character:** For manipulating text strings.
- **Number:** For mathematical calculations.
- **Date:** For handling date and time data.
- **Conversion:** For changing data from one type to another (e.g., String to Number).
- **General:** For utility operations and handling null values.

2. Character Functions: Case Manipulation

Case manipulation functions modify the capitalization of alphabetic characters.| Function | Conversion Rule | Example Input | Result || ------ | ------ | ------ | ------ || UPPER | Converts all alphabets to capital letters. | 'shri ganesh' | 'SHRI GANESH' || LOWER | Converts all alphabets to small/lower case. | 'SHRI GANESH' | 'shri ganesh' || INITCAP | Converts the first alphabet of each word to capital and the rest to small. | 'shri ganesh' | 'Shri Ganesh' |
The Word Separator Rule In Oracle SQL, INITCAP identifies "words" based on special characters, not just spaces. For example, INITCAP('AMar5akBaR^anTHonY') results in Amar5akbar^Anthony. Note that the special character ^ triggered the capitalization of the following 'A', but the digit 5 did **not** act as a separator.**Classification of Characters** Functions interact with three distinct types of characters:

1. **Alphabets:** A-Z and a-z.
2. **Digits:** 0-9.
3. **Special Characters:** All remaining symbols (e.g., #, $, ^, and spaces).

3. Character Functions: Character Manipulation

These functions allow for the structural modification and analysis of strings.

Concatenation

- **CONCAT(arg1, arg2) :** Joins exactly two arguments. To join more than two strings, you must nest the functions (e.g., CONCAT(arg1, CONCAT(arg2, arg3))).
- **|| Operator:** Performs the same task as CONCAT but is more flexible, allowing for the joining of multiple strings without nesting (e.g., 'Hello' || ' ' || 'World').

Padding (LPAD and RPAD)

Used to add characters to the left or right of a string to reach a specified length.

- **Arguments:** Accepts 2 or 3 arguments. If the 3rd argument (the padding character) is omitted, it defaults to a space.
- **Truncation Rule:** If the specified length is shorter than the input string, the function will truncate the string. **Crucially, both LPAD and RPAD truncate from the right.** For example, LPAD('Alexander', 6) results in 'Alexan'.

Trimming (LTRIM, RTRIM, and TRIM)

- **LTRIM / RTRIM :** Removes specified characters or combinations from the left or right side respectively.
- **TRIM :** Removes characters from the LEADING, TRAILING, or BOTH sides (defaults to BOTH).
- **Limitation:** TRIM only allows a **single character** to be specified. Attempting to trim multiple characters (e.g., TRIM('01' FROM ...)) will result in an error.

Length and Search

- **LENGTH :** Returns the number of characters in a string. For date types, the output is sensitive to NLS_DATE_FORMAT. For example, a format of 'DD-MON-RR' returns a length of 9, while 'DD-MONTH-RRRR' returns a greater length.
- **INSTR :** Returns the numeric position of a search string. It supports up to four arguments: INSTR(string, search_string, start_position, occurrence).
- *Example:* INSTR('NEELKAMAL', 'L', 3, 2) finds the *second* occurrence of 'L' starting from the third character.

Transformation: TRANSLATE vs. REPLACE

- **REPLACE :** Performs word-by-word substitution. If the third argument is omitted, it removes the specified word.
- **TRANSLATE :** Performs character-by-character substitution and requires exactly 3 arguments.
- **The Deletion Rule:** If a character is present in the second argument but has no corresponding mapping in the third argument, it is **deleted** . For example, TRANSLATE(' R A M ', '# ', '#') results in 'RAM' because the space was present in the second argument but not the third.

Extraction (SUBSTR)

- **Logic:** SUBSTR(string, start_position, length) extracts a portion of a string.
- **Indices:** A positive index starts from the left (1 is the first character). A negative index starts from the right.
- **Boundary Cases:** If the length argument is **0, negative, or NULL** , the function returns **NULL** .

4. Number Functions

Power and Root

- **POWER(base, exp) :** Finds the raised value. A fractional exponent (e.g., 0.5 or 1/2) calculates the square root.
- **Error Note:** Using a negative base with a fractional exponent results in ORA-01428: argument out of range.
- **SQRT(n) :** Returns the square root of a positive number. Negative inputs trigger a range error.

Remainder and Integers

- **MOD(n1, n2) :** Returns the remainder of a division.
- **FLOOR(n) :** Returns the current or previous lower integer.
- **CEIL(n) :** Returns the current or next higher integer.

Rounding and Truncating Numbers

Using the value **4729.38** as an example:

- **TRUNC(n, precision) :** Removes digits. TRUNC(4729.38, -2) results in 4700 (it zeros out digits to the left of the decimal).
- **ROUND(n, precision) :** Rounds based on the "5 or higher" rule. ROUND(4729.38, -3) results in 5000 because the '7' in the hundreds place rounds up.

5. Date Functions and Arithmetic

Basic Arithmetic

- **Weeks:** To add or subtract weeks, use 7-day intervals (e.g., SYSDATE + 21 for three weeks).

Specialized Functions

- **ADD_MONTHS(date, n) :** Adds/subtracts months. This is used for quarters (3), years (12), decades (120), and centuries (1200).
- **Century Note:** While ADD_MONTHS can handle centuries, the results may look identical unless the NLS_DATE_FORMAT is set to RRRR or YYYY to display the full four-digit year.
- **MONTHS_BETWEEN(d1, d2) :** Returns the months between two dates.
- **LAST_DAY(date) :** Returns the last date of the input's month.
- **NEXT_DAY(date, 'day') :** Returns the date of the next specified weekday. The day string must be at least three characters (e.g., 'WED').

6. Advanced Date Logic: Rounding and Truncating Dates

Truncation Logic

TRUNC with dates always returns the **first date** of the specified interval:

- **'MONTH'** : Returns the 1st of the current month.
- **'Q' (Quarter)** : Returns the 1st day of the first month of the quarter (Jan 1, Apr 1, Jul 1, or Oct 1).
- **'YEAR'** : Returns January 1st.

Rounding Logic

ROUND determines the result based on whether the date has passed the "Halfway" point:| Interval | First Half (Returns Current Start) | Second Half (Returns Next Start) || ------ | ------ | ------ || **Month** | Days 1–15 | Day 16 to End of Month || **Quarter** | 1st of Quarter to 15th of Middle Month | 16th of Middle Month to End of Quarter || **Year** | January 1 – June 30 | July 1 – December 31 |
Comparative Logic: To identify employees hired in the **first half** of a period, you can use the condition: WHERE ROUND(HIRE_DATE, 'MONTH') = TRUNC(HIRE_DATE, 'MONTH').

7. Rules for Nested Functions

Nesting involves placing one function inside another as an argument.

1. **Infinite Limits:** Functions can be nested to an infinite level.
2. **Compatibility:** Inner and outer functions can be of the same type or different types.
3. **Execution Order: Execution always proceeds from the innermost function to the outermost function.**
4. **Display:** Only the result of the outermost function is displayed.
5. **Input/Output Flow:** The result of each inner function serves as the input for its immediate outer function.

8. Practical Applications & Exercises (Student Self-Review)

Use the following logic-based solutions to verify your understanding:

- **Joining Month Tenure:** To calculate days worked including the hire date: LAST_DAY(HIRE_DATE) - HIRE_DATE + 1
- **3rd Monthly Salary Date:** If paid on the 1st of the month: ADD_MONTHS(LAST_DAY(HIRE_DATE) + 1, 2)
- **4th Weekly Salary:** If paid every Saturday: NEXT_DAY(HIRE_DATE, 'SAT') + 21
- **Hiring Trends:** To find those hired in the second half of a month: WHERE ROUND(HIRE_DATE, 'MONTH') != TRUNC(HIRE_DATE, 'MONTH')
- **Tenure Tracking:** Identify employees with over 30 years of service: WHERE MONTHS_BETWEEN(SYSDATE, HIRE_DATE) / 12 > 30